HandCAPTCHA: A CAPTCHA You Solve with Your Hand

1. Project Title and Team Information

Item	Details
Project Title	HandCAPTCHA: A Spot-the-Difference CAPTCHA Solved by Hand
Course	Introduction to Computer Vision
Team Name	HandCAPTCHA
Team Members	최민석, 전영민, 정지훈, 이채민, 정승호

2. Project Description

Problem. Conventional image CAPTCHAs require only mouse clicks, so they can be easily bypassed with image recognition models and automated clicking tools. HandCAPTCHA is a CAPTCHA that users pass by physically moving their hand in front of a webcam to perform a required action, making it impossible to solve through mouse automation alone.

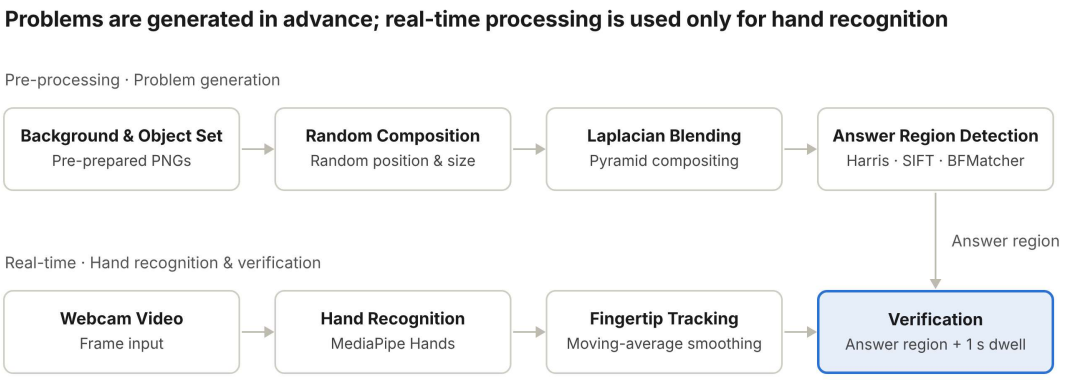
Input.

- Real-time webcam video: the user's hand movements
- Pre-prepared background images (about 5) and object images (about 10 transparent-background PNGs): used for problem generation

Expected Output.

- A side-by-side spot-the-difference problem screen created through random composition and Laplacian blending
- A fingertip cursor and hand-landmark overlay on the problem screen, with a dwell progress gauge
- Verification result: highlight of the answer region and a pass/fail indicator
- Auxiliary screen: visualization of Harris corners and SIFT/ORB keypoints (color-coded by whether a correspondence exists) and the detected answer region (a before/after blending comparison will be included as images in the presentation slides)

Processing Pipeline



System overview · 4 problem-generation stages, 4 real-time processing stages

Once a problem image is generated, Harris corners and SIFT/ORB keypoints are compared between the left and right images to find the answer region, and verification compares this region with the fingertip coordinates. If verification fails, a new problem is presented.

3. Technical Components

We use the following techniques covered in class: **Laplacian pyramids**, **Harris corner detection**, **SIFT/ORB**, **BFMatcher**, and **Canny edge detection**. Feature detection (Harris, SIFT/ORB) and matching (BFMatcher) are used to automatically find the answer region.

3-1. Problem Generation: Laplacian Pyramid Blending

Technique	OpenCV Function	Purpose
Laplacian pyramid blending	pyrDown, pyrUp	Seamlessly composite the boundary between the background and the object
Random placement	resize, warpAffine	Randomize the object's position and size, and apply a slight rotation and scaling to the entire right image

3-2. Answer Region Detection: Harris Corners and SIFT/ORB Keypoints

Stage	Technique	OpenCV Function	Purpose
1. Candidate selection	Harris corner detection	cornerHarris	Compare the corner distributions of the left and right images and select regions with newly appearing corners as candidates
2. Keypoint extraction	SIFT / ORB	SIFT_create, ORB_create	Extract keypoints and descriptors from the left and right images
3. Correspondence search	BFMatcher + ratio test	BFMatcher, knnMatch	Confirm the region where right-image keypoints with no left-image correspondence cluster as the answer region
4. Contour display	Canny edge detection	Canny	Extract the contour of the object within the answer region and highlight it on the result screen

Because the right image is slightly rotated and scaled, a direct pixel-by-pixel comparison cannot locate the difference. SIFT/ORB features are robust to rotation and scale changes, so they can still match the same points under these conditions; this is why we use SIFT/ORB. Keypoints with and without correspondences are drawn in different colors (drawKeypoints) on the auxiliary screen.

3-3. Hand Recognition and Verification

Technique	Purpose
MediaPipe Hands (pretrained model)	Extract index-fingertip coordinates from hand landmarks
Coordinate smoothing (moving average)	Reduce cursor jitter caused by hand tremor and motion blur
Dwell judgment	Treat the answer as correct when the fingertip stays in the answer region for about 1 second

4. Expected Demo Method

Input. We will run a live demo using a real-time webcam and, for a stable presentation, also submit a pre-recorded demo video of the same flow.

Demo Flow.

1. Launch the program → a problem is generated from a random combination of a background and an object
2. The side-by-side problem and an instruction ("Point to the part that is different") are displayed
3. As the user moves their hand in front of the webcam, the fingertip cursor follows on screen
4. Holding on the differing spot for about 1 second counts as correct → pass indicator; if wrong, a new problem is presented

5. The auxiliary screen shows Harris corners, SIFT/ORB keypoints, and the answer-region detection process to explain the techniques

Verification Methods. The main method is pointing to the difference; if time permits, the methods below will be added.

Method	Action	Type
Point to the difference	Hold the index fingertip on the answer region for about 1 second (dwell)	Required
Hand slider	Move the hand left and right to slide the object into its correct position	Optional
Drag and drop	Close the hand to grab a shape, then open it to release	Optional

Using dwell instead of a click gesture reduces the burden of finger-gesture recognition. Facial expression recognition is excluded because it is unnecessary for the purpose of a CAPTCHA.

5. Team Roles

Role	Responsibilities	Member
CV Processing 1 (Problem Generation)	Random background/object composition, Laplacian pyramid blending, rotation and scaling of the right image	전영민
CV Processing 2 (Answer Region Detection)	Harris corner candidate selection, SIFT/ORB keypoint extraction, BFMatcher matching, Canny contour display	정승호
Hand Recognition	MediaPipe Hands integration, index-fingertip coordinate extraction, moving-average smoothing	최민석
Verification	Fingertip–answer-region comparison, 1-second dwell timer, pass/fail handling, and new problem generation	정지훈
GUI Implementation	Problem screen, fingertip cursor, dwell gauge, result screen, auxiliary screen, and overall integration	이채민